MCP and tool calling

From retrieval to agents with tools and session memory

Lecture 3 connects the retrieval foundation from Lecture 2 to an agent that can choose what to do next. We expose Python functions and managed MCP tools through one interface, use Genie for questions about structured data, and persist conversation history in Lakebase.

What you should be able to do

- Explain the difference between a predefined workflow and a model-directed agent.
- Define a tool with a name, description, input schema, and executable function.
- Trace the complete tool-calling loop, including assistant messages and tool results.
- Discover Genie and Unity Catalog function tools through MCP.
- Save and restore session messages, and explain which identity accesses the database.

Reading map

Pages	Topic
2–3	Orchestration and the agent execution loop
4–5	Custom tools and retrieval with Lakebase pgvector
6–8	MCP discovery, integration, and Unity Catalog functions
9–10	Genie configuration and conversations
11–13	Persistent memory, connections, and authentication
14–16	Troubleshooting, homework, and implementation map

Before starting: use your course environment configuration and the Lecture 2 Lakebase retrieval setup. The examples use the repository's pinned dependencies and existing resources. Full agent packaging, MLflow tracing, and deployment follow in the next lecture.

The practical architecture combines a local `search_papers` tool, managed Genie tools, and a separate memory store. MCP is one way to expose a capability; it is not required for every function in an agent.

Choose the orchestration deliberately

An agent combines instructions, a model, tools, and runtime behavior. The model proposes an answer or a tool call; application code controls what can execute, manages state, and decides when to stop. Definitions vary, but this division is the one used in the course.

Design	Who chooses the next step?
Predefined workflow	The developer fixes the sequence and branches in application code. LLM calls can still appear inside that sequence.
Model-directed agent	The model chooses among the tools exposed by the application and may make multiple calls before answering.
Combined design	A model routes the request; a controlled workflow handles a known process.

Simple RAG and agentic RAG

Simple RAG

Question → retrieve chunks → build prompt → model → answer

Agentic RAG

Question → model → optional search tool → model → answer or another tool

Notebook 3.1b always retrieves documents before generation. Notebooks 3.1 and 3.2 expose retrieval as a tool, allowing the model to choose the query, result count, and optional year filter. An agent can skip retrieval for a greeting or use a calculator for arithmetic.

Flexibility has a cost

Use a fixed workflow when the required sequence is already known. Dynamic tool selection is useful for varied requests, but introduces additional model calls, latency, and failure paths. Giving a model a tool list does not make every part of the application autonomous.

Sketch the course agent

- Research evidence: search paper chunks through Lakebase pgvector.
- Structured analysis: ask Genie about paper metadata, counts, dates, or authors.
- Conversation continuity: load previous messages using the session ID.
- Runtime controls: bound iterations, validate calls, handle errors, and record execution.

Read the tool-calling loop

The model does not execute a Python function. It returns a structured request containing the tool name, arguments, and a call ID. The application resolves the name in an approved registry, executes the function, and returns the result to the model.

1. Send instructions, conversation messages, and tool schemas to the model.
2. If the assistant returns no tool calls, return its answer.
3. Append the assistant message containing the tool calls to the conversation.
4. Parse and validate each call's arguments, then execute the selected function.
5. Append one tool result for each call, using the matching `tool_call_id`.
6. Call the model again with the updated messages, within the iteration budget.

```
# Core pattern from SimpleAgent in notebook 3.2.
# Run inside its chat loop after appending the assistant message.
for call in assistant_message.tool_calls:
    args = json.loads(call.function.arguments)
    result = self.tools[call.function.name].exec_fn(**args)
    messages.append({
        "role": "tool",
        "tool_call_id": call.id,
        "content": str(result),
    })
```

Preserve the conversation structure

A tool result must follow the assistant message that requested it. Preserve every call ID, especially when one assistant response requests several tools. Appending only the result text loses the relationship between request and execution.

Bound work and handle failure

The teaching loops use `max_iterations=10`. That limits model iterations, not total tool calls: a single response may request several tools. Production controls can also bound total calls, output size, elapsed time, and polling. Return a clear limit result rather than presenting an unfinished run as a complete answer.

Teaching-code boundary: notebook 3.2 directly parses JSON, looks up names, and executes calls. Add validation and controlled error responses before relying on this loop in an application. Notebook 3.1 catches execution errors, but argument parsing still happens outside that handler.

Define a useful custom tool

A tool needs both a model-facing contract and an executable implementation. `ToolInfo` in `src/arxiv_curator/mcp.py` holds `name`, `spec`, and `exec_fn`. The model receives the specification; the application keeps the callable.

```
from arxiv_curator.mcp import ToolInfo

def add_numbers(a: float, b: float) -> float:
    return a + b

add_tool = ToolInfo(
    name="add_numbers",
    spec={
        "type": "function",
        "function": {
            "name": "add_numbers",
            "description": "Add two numbers and return their sum.",
            "parameters": {
                "type": "object",
                "properties": {
                    "a": {"type": "number"},
                    "b": {"type": "number"},
                },
                "required": ["a", "b"],
                "additionalProperties": False,
            },
        },
    },
    exec_fn=add_numbers,
)
assert add_tool.exec_fn(a=10, b=3) == 13
```

This is the nested function schema used by the course's Chat Completions calls. Do not assume every API uses the same tool envelope. The `ToolInfo` docstring mentions Responses, but the actual course specification and calls use Chat Completions.

Design the contract for selection and execution

- Describe when the tool is useful and what it returns. Distinguish overlapping tools.
- Document parameters and required fields; make defaults agree with the callable.
- Validate values in application code. A schema and Python type annotations alone do not enforce safe execution.
- Return compact structured results and actionable errors. Test the function directly before testing model selection.

Notebook 3.1 develops a calculator, a paper search tool, and a registry. A local registry is a name-to-function mapping; it becomes an MCP server only when the protocol is actually implemented.

Expose retrieval as a tool

The course uses `LakebasePgVectorManager.hybrid_search()` to combine vector similarity with keyword ranking. Both simple RAG and the agentic version can reuse this retrieval implementation; the difference is who decides when and how to call it.

```
# In notebook 3.2, after loading cfg and creating WorkspaceClient w:
pgvector = LakebasePgVectorManager(config=cfg, workspace=w)

# The notebook's search_papers wrapper interprets year_filter.
result = search_papers(
    query="transformer attention mechanisms",
    num_results=3,
    year_filter="2025-2026",
)
```

Input or output	Meaning in notebook 3.2
`query`	Required search text. The model may rewrite the user's question for retrieval.
`num_results`	Number of requested chunks; defaults to 5. Validate an acceptable range.
`year_filter`	Optional year or range, such as 2026 or 2025-2026. The wrapper converts these to numeric predicates.
Returned JSON	Title, arXiv ID, authors, year, score, and an excerpt truncated to 500 characters.

What the result can and cannot support

The search returns chunks, not necessarily distinct papers. Several chunks may describe one paper. Preserve identifiers for citations, deduplicate when appropriate, and avoid claiming to summarize a full paper from a short excerpt.

During the lecture, some questions were poorly supported by the small dataset. A fluent response is not proof that retrieval succeeded. Inspect the returned evidence, use an explicit insufficient-evidence response, and evaluate any relevance threshold against representative questions.

Compare with the fixed RAG baseline

Notebook 3.1b implements `retrieve_documents()`, `build_rag_prompt()`, and `rag_query()`. Its `SimpleRAG` class also stores a Python list of conversation messages. That history lives only in the running process and does not provide persistence across independent serving requests.

Keep content and control separate: retrieved passages and tool outputs are evidence. Instructions found inside them must not override the application's tool permissions or trusted instructions.

Understand what MCP standardizes

Model Context Protocol provides a common interface for an AI application to discover and use server capabilities. The host application manages MCP clients; a client communicates with a server. MCP does not choose your orchestration or replace your agent loop.

Capability	Purpose
Tools	Callable operations, such as searching papers or asking a data question.
Resources	Context or information that a client can retrieve.
Prompts	Reusable templates exposed by a server.

Agent application → MCP client → MCP server → underlying capability
Discovery returns tool names, descriptions, and input schemas.
Invocation returns results that the application can send to the model.

Local functions and MCP

A local function gives direct control over custom logic and is easy to test. MCP is useful when a capability needs a standard interface across clients or is already offered by a platform. Local functions can also be shared through libraries or services; MCP standardizes that access rather than making reuse possible for the first time.

Databricks endpoints in this course

Notebook 3.2 uses workspace MCP endpoints for a curated Genie space and a Unity Catalog function. These use underlying resource permissions and bypass Unity Gateway. Databricks also provides `system.ai` MCP services through Unity Gateway, with MCP-level grants and policies. These are different access paths.

```
# Course workspace endpoints
{host}/api/2.0/mcp/genie/{genie_space_id}
{host}/api/2.0/mcp/functions/{catalog}/{schema}/{function_name}

# Unity Gateway service pattern
{host}/ai-gateway/mcp-services/{mcp_name}
```

Managed capabilities include analytics, SQL, functions, and search. The course keeps pgvector retrieval as a custom function. External servers and servers hosted on Databricks Apps are extension options; they are not needed for the exercises.

Read more MCP architecture

Read more Databricks MCP services and workspace endpoints

Discover and adapt MCP tools

The course adapter converts the server's tool definitions into the same `ToolInfo` objects used for local functions. This lets one agent select from both types of tool without changing its execution loop.

```
from databricks.sdk import WorkspaceClient
from databricks_mcp import DatabricksMCPClient
from arxiv_curator.mcp import create_mcp_tools

w = WorkspaceClient()
genie_url = (
    f"{w.config.host.rstrip('/')}/api/2.0/mcp/genie/"
    f"{genie_space_id}"
)
client = DatabricksMCPClient(
    server_url=genie_url, workspace_client=w
)
for tool in client.list_tools():
    print(tool.name, tool.description, tool.inputSchema)

# Notebook with top-level await; use the deployed space ID.
genie_tools = await create_mcp_tools(w, [genie_url])
tools = [search_papers_tool, *genie_tools]
tools_by_name = {tool.name: tool for tool in tools}
```

How the adapter works

1. Call `list_tools()` for each configured server.
2. Copy each tool's `inputSchema` into the model-facing `parameters`.
3. Keep the tool name and description supplied by the server.
4. Bind an execution wrapper to the server URL and tool name.
5. Invoke `client.call_tool(tool_name, kwargs)` when the model selects that tool.

Async declaration versus asynchronous execution

The repository declares `create_mcp_tools()` with `async def`, so callers await it or use an event-loop runner. Its body calls synchronous SDK methods and contains no `await`. The declaration alone does not make those network calls concurrent. Notebook 3.2 uses `nest_asyncio` and `asyncio.run()`.

Adapter limits: the wrapper joins `c.text` from every result block. That assumes text-only output and does not explicitly check MCP error flags. A broader adapter should handle content types and errors, preserve result structure, and reject duplicate tool names rather than silently overwriting them in a dictionary.

Use governed Unity Catalog functions

Notebook 3.1c registers four arithmetic functions in the configured catalog and schema. The same function can be called from SQL or exposed through the managed functions MCP endpoint. The example makes discovery and permissions easy to see without a complex business function.

```
# In notebook 3.1c, after loading cfg and creating spark:
function_name = f"{cfg.catalog}.{cfg.schema}.add_numbers"
spark.sql(f"""
    CREATE OR REPLACE FUNCTION {function_name}(a DOUBLE, b DOUBLE)
    RETURNS DOUBLE
    LANGUAGE PYTHON AS
    $$
    return a + b
    $$
""")
spark.sql(f"SELECT {function_name}(10, 3) AS result").show()
```

```
# In notebook 3.2, after creating w:
uc_url = (
    f"{w.config.host.rstrip('/')}/api/2.0/mcp/functions/"
    f"{cfg.catalog}/{cfg.schema}/add_numbers"
)
uc_tools = await create_mcp_tools(w, [uc_url])
add = next(t for t in uc_tools if t.name.endswith("add_numbers"))
print(add.exec_fn(a=10, b=3))
```

Register first, then discover

Run the function-creation notebook before the UC section of notebook 3.2. The two activities are in separate jobs, so running the Genie/MCP job alone does not create the arithmetic functions. Use your configured catalog and schema rather than a teacher's names.

Governance follows the executing identity

The caller needs the relevant catalog, schema, and function privileges. Being able to see a function in one UI session does not establish that another job identity can execute it. Test direct SQL execution and MCP invocation separately to narrow down a failure.

Keep the teaching functions small. For more complex UDFs, verify runtime, dependency, and networking support rather than assuming arbitrary notebook code will execute unchanged. Changes through `CREATE OR REPLACE` also deserve source control and compatibility checks for existing callers.

The division example returns SQL null when dividing by zero. A tool contract should explain exceptional results so the agent can distinguish "no value" from a successful calculation.

Prepare a useful Genie space

The lecture calls curated spaces **Genie Agents**; the SDK and bundle still use “space”. Genie Code assists with code, and Genie One is a broader analytics experience. Genie translates natural-language questions about structured data into SQL and results. In this course it complements chunk retrieval: use Genie for “Which authors have the most papers?” and the search tool for evidence about a method described inside papers.

Define the space with the bundle

`resources/genie_space.yml` declares `arxiv-curator-space`, references the configured SQL warehouse, and includes `catalog.schema.arxiv_papers`. Its `serialized_space` is a structured YAML object with `version: 2`. The current repository does not require a separate JSON configuration file.

```
# Structural excerpt from resources/genie_space.yml
resources:
  genie_spaces:
    arxiv-curator-space:
      title: arxiv-curator-space
      warehouse_id: "${var.warehouse_id}"
      serialized_space:
        version: 2
        # Full file includes config, data_sources, and instructions.
```

Give Genie the meaning of the data

Course metadata	Why it matters
`published` and `processed`	BIGINT values in yyyyMMddHHmm format. Convert before date arithmetic; they are not native timestamps.
`authors`	An array of names. Explode to count by author; join for display.
“Latest papers”	Order by publication time, not ingestion time.
Examples and descriptions	Specify what one row represents, useful columns, sample questions, and correct SQL.

The bundle includes examples for recent papers and author counts. Good table and column comments belong in the data preparation workflow too; they help people and downstream tools interpret the same data consistently.

Use the deployed ID

`resources/lecture3_tools.yml` passes `${resources.genie_spaces.arxiv-curator-space.id}` as the `genie_space_id` parameter. Interactive notebooks have a hard-coded dev fallback; override it with your own deployed space ID. A stale fallback can cause permission errors or query the wrong dataset.

The repository requires a recent Databricks CLI in `databricks.yml`. Validate the bundle against that requirement before deployment. Reuse the course warehouse and provisioned resources rather than creating a new warehouse for this exercise.

Read more Bundle resource reference

Keep Genie conversations straight

Notebook 3.2b demonstrates the SDK conversation flow. A conversation ID groups questions and follow-ups; a message ID identifies a particular request and its response.

```
# genie_space_id is supplied by the job or your notebook widget.
space = w.genie.get_space(
    space_id=genie_space_id,
    include_serialized_space=True,
)
conversation = w.genie.start_conversation_and_wait(
    space_id=space.space_id,
    content="Find the last 10 papers published",
)
message = w.genie.create_message_and_wait(
    space_id=space.space_id,
    conversation_id=conversation.conversation_id,
    content="Return the list of authors of those papers",
)
print(message.as_dict())
```

Query and poll are different operations

The Genie MCP interface exposes query and response-polling capabilities. Discover their exact names and schemas with `list_tools()`. The query operation submits a question; polling uses the returned identifiers to retrieve progress or results for that message.

Polling does not add a follow-up question. To continue a conversation, submit another question with the conversation ID. Poll the resulting message if it is not complete. The SDK's `_and_wait` methods hide the waiting step from the example.

Inspect the answer and the SQL

Check the query, filters, aggregation, and returned rows. A correct-looking sentence can still be based on the wrong date field or a mistaken join. If Genie is slow or inaccurate, review the table scope, metadata, examples, and warehouse behavior before adding more orchestration. Verify current workspace quotas and handle rate-limit responses; do not assume unlimited calls.

The lecture also demonstrated an external assistant using Databricks MCP tools to build a dashboard artifact. That was an illustration of reusing tools across applications, not an additional notebook deliverable.

Persist history outside the process

An endpoint request does not automatically retain the previous turn. A caller supplies a session identifier, and the application loads stored messages before generating the next response. Lakebase provides a PostgreSQL store for this history while application instances can remain stateless.

Request + session ID → load history → add new user message
Run the tool loop → save new messages → return answer
Next request with the same session ID → restore that history

A minimal session table

```
CREATE TABLE IF NOT EXISTS session_messages (
    id SERIAL PRIMARY KEY,
    session_id TEXT NOT NULL,
    message_data JSONB NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_session_messages_session_id
ON session_messages(session_id);
```

`message_data` holds a message object, such as role and content. Decide whether to persist tool calls and results as well. If replaying tool messages, preserve their corresponding assistant calls and IDs so the restored conversation is valid.

Three kinds of stored information

Store	Purpose
Retrieval data	Research content and embeddings used to find evidence.
Session history	Messages used to continue a particular conversation.
Observability traces	Execution details used to investigate latency, failures, and behavior.

Durable user memory is another layer: a background process could summarize and deduplicate facts across sessions. The lecture discusses that possibility, but the current memory helper only appends and loads raw session messages.

A session ID is not an authorization check. A multi-user application must verify that the current caller may read or append to that session. The teaching table has no user or tenant ownership field, so this must be designed before sharing the application with independent users.

Appending unlimited history also increases prompt size and cost. Select, trim, or summarize history deliberately, and verify that the required context and tool-call relationships survive.

Use the Lakebase memory helper

The current code uses a Lakebase project and the SDK's `PostgresAPI`. It discovers a branch and endpoint, obtains a short-lived database credential, and connects using TLS.

```
from uuid import uuid4
from databricks.sdk import WorkspaceClient
from arxiv_curator.config import get_env, load_config
from arxiv_curator.memory import LakebaseMemory

# spark is available in the course notebook environment.
cfg = load_config("../project_config.yml", get_env(spark))
w = WorkspaceClient()
memory = LakebaseMemory(
    project_id=cfg.lakebase_project_id, workspace=w
)
memory.ensure_schema()
session_id = f"lecture3-{uuid4()}"
new_messages = [
    {"role": "user", "content": "Find papers about attention."},
    {"role": "assistant", "content": "Which year should I search?"},
]
memory.save_messages(session_id, new_messages)
loaded = memory.load_messages(session_id)
assert len(loaded) == 2
```

Connections and credentials

The helper creates a client-side `psycopg_pool.ConnectionPool` lazily, with one to five connections. Calls borrow a connection and return it to the pool. This is separate from any managed database-side pooler.

On `psycopg.OperationalError`, load and save reset the pool and re-raise. A later use can obtain fresh credentials; the failed operation is not automatically retried. Other exceptions are logged: load returns an empty list and save returns without confirming persistence. Verify a read-back rather than treating a "Saved" notebook log as proof.

Know the teaching implementation's limits

- Without an explicit app endpoint, the helper selects the first returned branch and endpoint. That is not a guarantee of selecting the production branch.
- Messages are ordered by timestamp only. Equal timestamps need a stable tie-breaker such as `id` for reliable replay.
- Save appends rows. Re-saving all loaded history duplicates messages; retries need an idempotency strategy.

The SPN notebook assumes the table already exists. Run the schema setup first, then grant the runtime identity the database privileges it actually needs.

Identify who accesses each resource

Authentication identifies the caller; authorization determines what that caller may do. In the course, MCP and Lakebase can use different identities. Check each client explicitly when diagnosing a permission error.

Mode	Access model
User credentials	Interactive calls use the authenticated user's grants.
Service principal	Calls use the application identity's grants. Downstream access is not automatically restricted to each end user's own permissions.
On-behalf-of-user flow	A supported deployment propagates the caller's identity. This needs explicit configuration; it is not implied by using MCP.

Service principal credentials in the course

```
# Notebook 3.4 reads Unity Catalog secrets; never print them.
os.environ["LAKEBASE_SP_CLIENT_ID"] = dbutils.secrets.get(
    catalog=cfg.catalog, schema=cfg.schema, key="client_id"
)
os.environ["LAKEBASE_SP_CLIENT_SECRET"] = dbutils.secrets.get(
    catalog=cfg.catalog, schema=cfg.schema, key="client_secret"
)
os.environ["LAKEBASE_SP_HOST"] = w.config.host
memory = LakebaseMemory(project_id=cfg.lakebase_project_id)
```

Dedicated `LAKEBASE_SP_*` variables allow the memory client to use a service principal without changing the default workspace client used by MCP. The helper also accepts `DATABRICKS_*` fallbacks. Supply a complete, consistent credential set: a partial set can make identity selection disagree with the connection's authentication.

Two permission layers

Lakebase project permissions control infrastructure access. PostgreSQL roles and grants control access inside a branch's database. A project grant alone is not a grant on every schema or table. Check the role, database connection, schema usage, table privileges, and sequence access when inserting into a SERIAL-backed table.

The course provisions identities and secrets. The reader of a Unity Catalog secret needs the relevant secret privileges; possession of a secret then enables authentication as that principal.

Least privilege in practice: separate one-time table creation from runtime reads and inserts where possible. Broad teacher or administrator access can make a demo work while concealing a missing grant for the deployed identity.

Read more Lakebase roles and project permissions

Diagnose failures one layer at a time

Start with the tool itself, then the adapter, then model selection. This separates deterministic implementation problems from model behavior and makes the lecture's permission and retrieval failures easier to interpret.

Symptom	What to check
Tool call fails directly	Configuration, inputs, current identity, resource existence, and direct backend access.
Tool works, agent ignores it	Name and description, parameter schema, competing tools, and the actual assistant tool-call output.
Permission denied	Which workspace client is used? Is this a UC resource grant, Genie/warehouse access issue, or a Postgres role/table grant?
No useful research answer	Corpus coverage, returned chunks, filters, scores, excerpt truncation, and whether the answer's citations support its claims.
Genie response is pending	Poll the existing message using its IDs. Bound waiting; do not repeatedly submit the same question.
Memory appears empty	Correct project/branch/endpoint, schema initialization, session ID, and warnings from the helper. An empty list can also mask an error.
Duplicate or reordered messages	Append only new messages, make retries idempotent, and use a deterministic ordering key.

A small verification set

1. Call the calculator with normal values and a division-by-zero case.
2. Search with a normal query, a single year, a year range, malformed input, and a question outside the corpus.
3. Discover MCP tools and execute the UC addition function; then test a Genie query and follow-up.
4. Save two messages under one session ID and read them back through a new helper instance. Confirm a second session is separate.
5. Exercise the agent's stopping limit and inspect the complete request/tool-result sequence.

Inspect the cost of the full path

Count model iterations and tool calls, not just final answers. Retrieval can involve an embedding call and database work; Genie uses its data-query path; memory adds database operations. Lakebase compute, storage, and synchronization are separate considerations. The course returns to billing and operational monitoring later.

These checks are exercises for your Databricks environment. A local document build cannot establish that a student's resource permissions or remote notebooks will succeed.

Homework and review questions

Build a small tool-using design for your own use case. Keep your implementation in a separate project folder with its own configuration and bundle file so the course examples remain easy to compare and update.

Homework

1. **Identify the tools.** For each capability, state the inputs, outputs, required identity, and whether a local function or MCP is appropriate.
2. **Sketch orchestration.** Show fixed steps, model-directed choices, stopping conditions, and failure paths. A Markdown or HTML diagram is sufficient.
3. **Connect and test the tools.** Include custom functions and an MCP example, including Genie. Test the callable independently before using it through the agent.
4. **Persist session history.** Save messages in Lakebase and load them on the next turn with the same session ID. Demonstrate isolation from a second session.
5. **Submit a pull request.** Include your implementation, design choices, evidence from test runs, and known limitations.

What to include in the pull request

- A concise explanation of the user request your agent handles and why the chosen orchestration fits.
- Tool schemas, source/configuration files, and reproducible example questions.
- Evidence of tool selection, a successful result, a controlled failure, and persistent memory.
- An explanation of the user/SPN permissions and how session access is protected.

Check your understanding

Question	Expected reasoning
Does exposing tools force an agent to use them?	No. The model may answer directly unless the application imposes a different control.
Why keep the assistant tool-call message?	It links the tool result to the requested operation and call ID.
Is Genie polling a new question?	No. It retrieves the status or result of an existing message.
Does a connection pool create durable memory?	No. The database stores the messages; pooling reuses connections.
Does an SPN bypass governance?	No. Its grants are enforced, but they are the SPN's grants rather than each end user's.

Implementation and reading map

Use the existing notebooks as runnable demonstrations. The notes explain their behavior and distinguish teaching shortcuts from controls you would add to a shared application.

File	Purpose
`notebooks/3.1_custom_functions_tools.py`	Tool specifications, calculator, search, registry, and agent loop.
`notebooks/3.1b_simple_rag.py`	Fixed retrieval workflow and process-local history.
`notebooks/3.1c_uc_function_example.py`	Register and call UC arithmetic functions.
`notebooks/3.2_mcp_integration.py`	Combine pgvector and Genie tools; invoke a UC function through MCP.
`notebooks/3.2b_genie.py`	Load the deployed space and continue a Genie conversation.
`notebooks/3.3_session_memory_lakebase.py`	Direct SQL persistence and the memory helper.
`notebooks/3.4.spn_authentication_lakebase.py`	Use UC secrets for SPN-based memory access.
`src/arxiv_curator/mcp.py` `src/arxiv_curator/memory.py`	Reusable tool adapter and session persistence.
`resources/genie_space.yml` `resources/lecture3_tools.yml`	Space definition, deployed ID, notebook jobs, and environment.